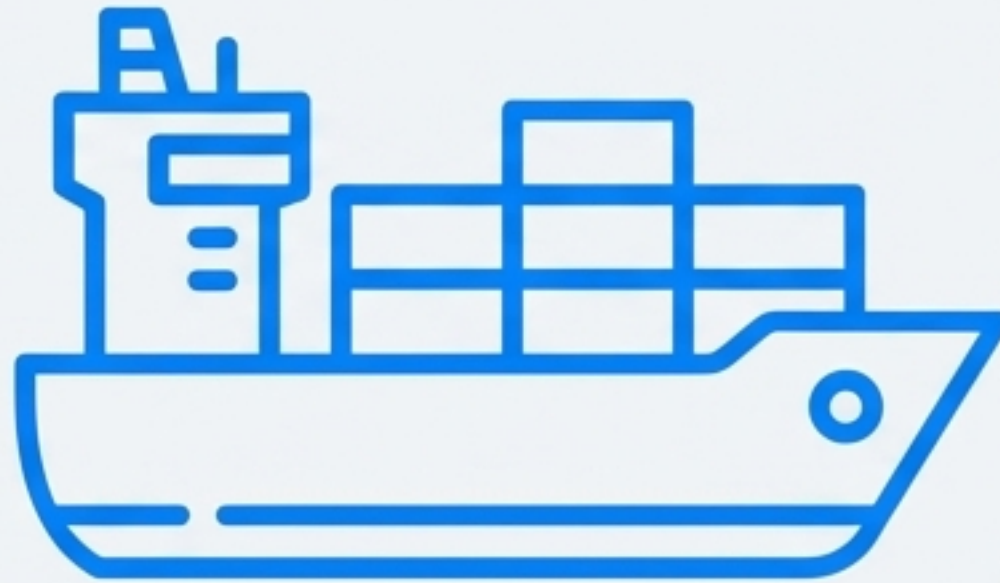


TP Interactif : Le Cycle de Vie d'un Conteneur Docker

Une introduction pratique aux commandes essentielles.



Phase 1 : Préparation et État des Lieux

Comment vérifier que Docker est correctement installé ?

```
docker --version
```

```
$ docker --version  
Docker version 20.10.17, build 100c701
```

Cette commande est le point de départ. Elle permet de confirmer que le moteur Docker est installé, fonctionnel et de connaître sa version. C'est la première étape avant toute opération.

Phase 1 : Préparation et État des Lieux

Comment lister les images disponibles localement ?

`docker images`

```
$ docker images
REPOSITORY    TAG                IMAGE ID           CREATED            SIZE
```

Une image Docker est un modèle en lecture seule qui contient les instructions pour créer un conteneur. Cette commande affiche toutes les images (les "plans de construction") que vous avez sur votre machine. Pour l'instant, la liste est probablement vide.

Phase 1 : Préparation et État des Lieux

Comment voir tous les conteneurs (actifs et arrêtés) ?

```
docker ps -a
```

```
$ docker ps -a
```

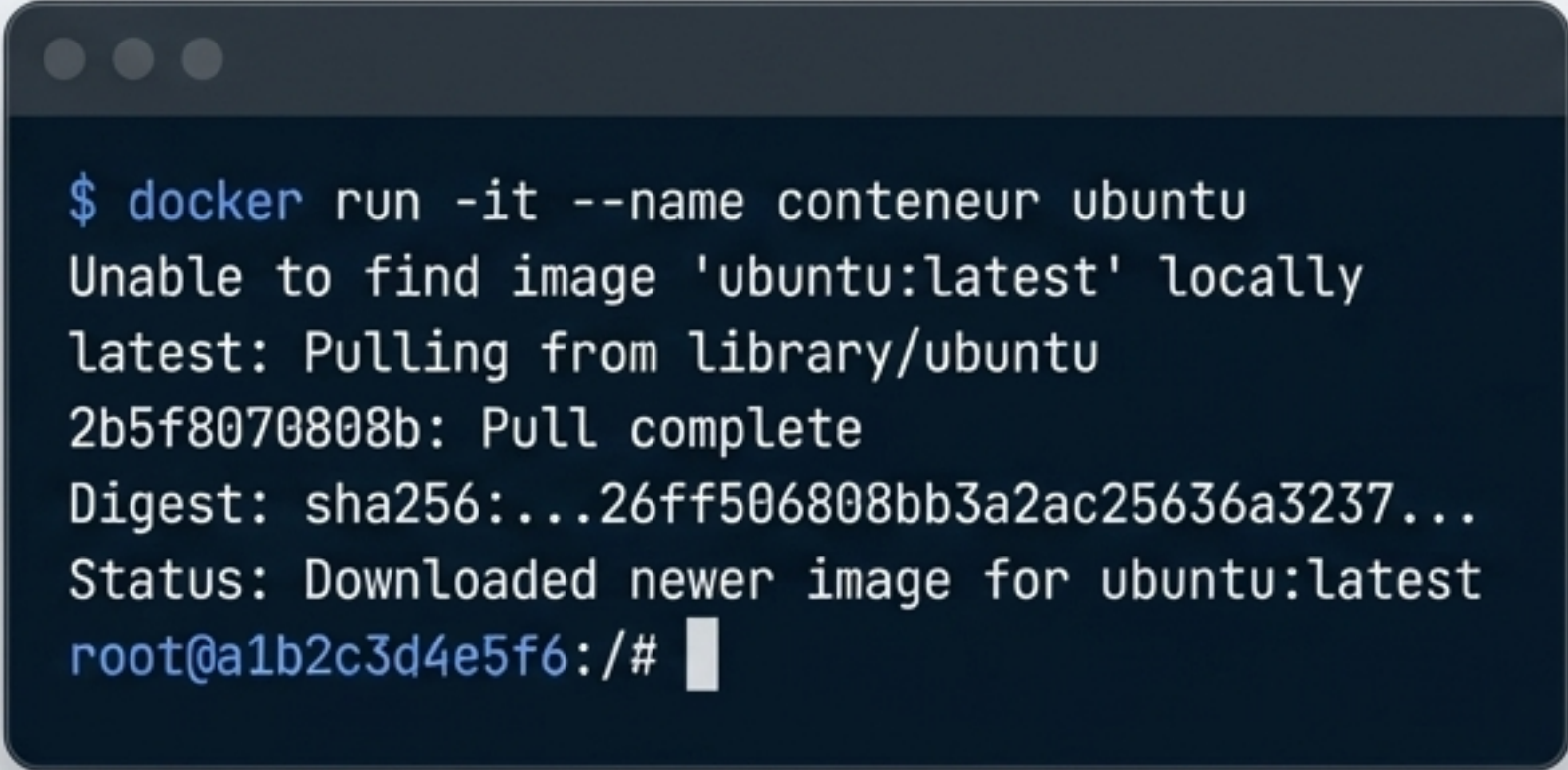
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

La commande `docker ps` seule ne montre que les conteneurs en cours d'exécution. L'option `-a` (pour 'all') est cruciale : elle révèle tous les conteneurs, y compris ceux qui sont arrêtés. Cela nous donne une vue complète de notre environnement.

Phase 2 : Naissance et Exploration

Comment créer et démarrer un conteneur en mode interactif ?

```
docker run -it --name conteneur ubuntu
```



```
$ docker run -it --name conteneur ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
2b5f8070808b: Pull complete
Digest: sha256:...26ff506808bb3a2ac25636a3237...
Status: Downloaded newer image for ubuntu:latest
root@a1b2c3d4e5f6:/#
```

C'est la commande magique. Décortiquons-la :

- **run** : Crée et démarre un nouveau conteneur.
- **-it** : Combine **-i** (interactif, garde STDIN ouvert) et **-t** (alloue un pseudo-TTY). Essentiel pour avoir un shell.
- **--name conteneur** : Donne un nom facile à retenir à notre conteneur.
- **ubuntu** : Spécifie l'image à utiliser. Docker la téléchargera si elle n'est pas présente localement.

Phase 2 : Naissance et Exploration

Comment installer des logiciels à l'intérieur du conteneur ?

apt update

```
root@a1b2c3d4e5f6:/# apt update
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Fetched 18.9 MB in 3s (6123 kB/s)
Reading package lists... Done
root@a1b2c3d4e5f6:/#
```

Une fois à l'intérieur de notre conteneur Ubuntu, nous pouvons utiliser le gestionnaire de paquets **apt**. La première étape est de mettre à jour la liste des paquets disponibles.

Phase 2 : Naissance et Exploration

Installons maintenant un éditeur de texte.

```
apt install nano
```

```
root@a1b2c3d4e5f6:/# apt install nano
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 13 not upgraded.
Need to get 421 kB of archives.
After this operation, 1,256 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Installing packages... Done
Installing package (ionuintiono... 0.1 (0/8) [18.3 kB]
Completing usernano 1.5.9[=====...] [48.2 kB]
Confirming nser packages... acmosved.
After this operation, 1,256 kB of pour cet environnement isolé.
nano [Y] you want to continue.
```

Avec la liste des paquets à jour, nous pouvons maintenant installer nano. Le conteneur va télécharger et configurer le logiciel, uniquement pour cet environnement isolé.

Phase 2 : Naissance et Exploration

Comment créer un fichier pour tester la persistance des données ?

```
nano test.txt
```



Utilisons l'éditeur que nous venons d'installer pour créer un fichier simple. Écrivez une phrase comme 'Hello Docker' à l'intérieur, puis sauvegardez (Ctrl+O, Enter) et quittez (Ctrl+X). Ce fichier est maintenant sauvegardé dans le système de fichiers du conteneur.

Phase 2 : Naissance et Exploration

Comment sortir de la session interactive du conteneur ?



exit

```
root@a1b2c3d4e5f6:/# exit  
$
```

La commande `exit` termine la session shell. Comme c'était le processus principal de notre conteneur interactif, le conteneur va s'arrêter. Nous sommes de retour sur notre machine hôte. Mais qu'est-il advenu de notre conteneur ?

Phase 2 : Naissance et Exploration

Où est passé notre conteneur ?

```
docker ps -a
```

```
> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a1b2c3d4e5f6	ubuntu	'bash'	2 minutes ago	[Exited (0) 5 seconds ago]		conteneur

En exécutant à nouveau cette commande, nous voyons maintenant notre conteneur nommé `conteneur` dans la liste. Son statut est "Exited". Il est arrêté, mais toutes nos modifications (comme l'installation de `nano` et la création de `test.txt`) sont sauvegardées en son sein. `docker ps` (sans -a) ne l'afficherait pas.

Phase 3 : La Seconde Vie

Comment redémarrer un conteneur qui a été arrêté ?

```
docker start conteneur
```

```
$ docker start conteneur  
conteneur
```

La commande `docker start` relance un conteneur arrêté en utilisant son nom ou son ID. Contrairement à `run`, elle ne crée pas un nouveau conteneur, mais redémarre l'existant. Le conteneur tourne maintenant en arrière-plan.

Comment vérifier que le conteneur est bien en cours d'exécution ?

```
docker ps
```

```
$ docker ps
```

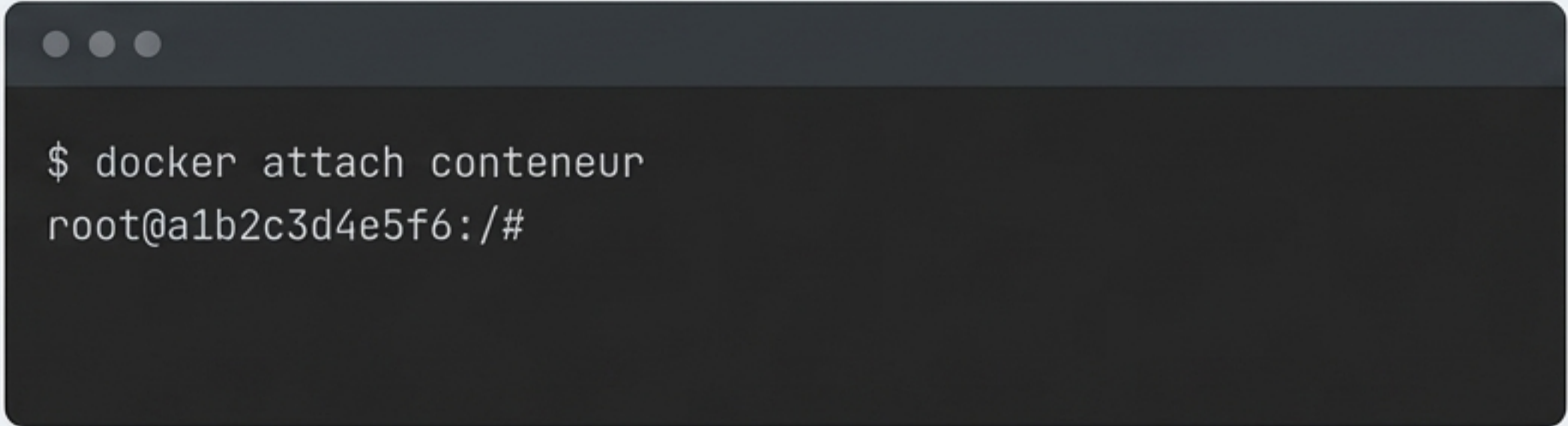
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a1b2c3d4e5f6	ubuntu	'bash'	3 minutes ago	[Exited (0) 5 seconds ago]		

Cette fois, sans l'option `-a`, notre conteneur apparaît ! Cela confirme qu'il est bien actif et tourne en arrière-plan (en mode 'detached'). Son statut est 'Up'.

Phase 3 : La Seconde Vie

Comment se reconnecter à un conteneur qui tourne en arrière-plan ?

```
docker attach conteneur
```



```
$ docker attach conteneur  
root@a1b2c3d4e5f6:/#
```

`docker attach` permet de se 'brancher' au processus principal d'un conteneur déjà en cours d'exécution. Nous sommes de nouveau à l'intérieur de notre shell.

Phase 3 : La Seconde Vie

Comment vérifier que nos modifications ont été conservées ?

```
cat test.txt
```

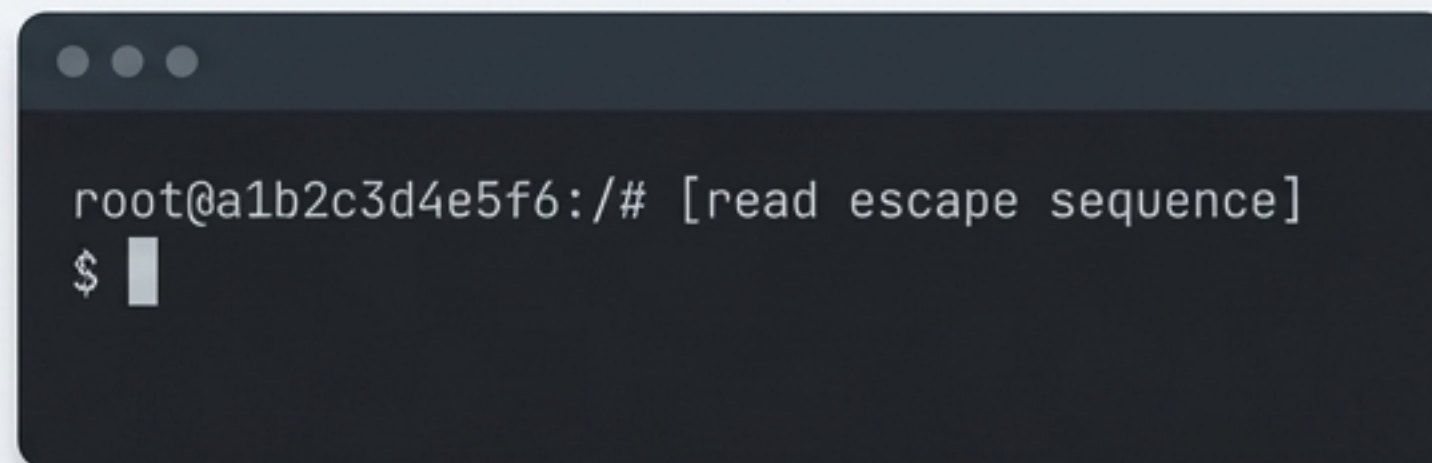
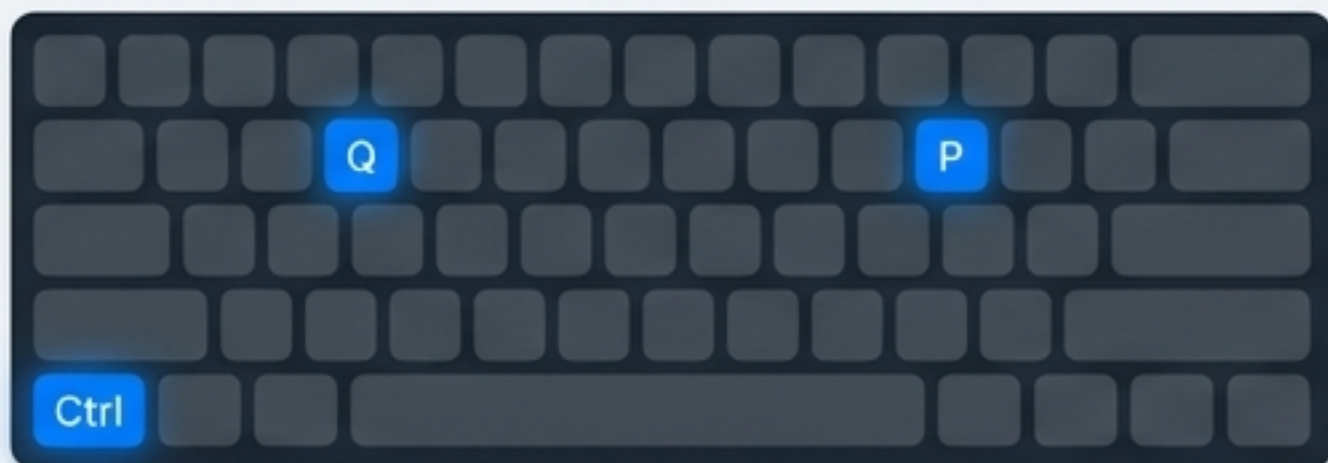


```
root@a1b2c3d4e5f6:/# cat test.txt  
Hello Docker
```

La commande ``cat`` affiche le contenu d'un fichier. En l'utilisant sur ``test.txt``, nous pouvons voir que le texte 'Hello Docker' que nous avons écrit est toujours là. L'état du conteneur a été parfaitement préservé entre l'arrêt et le redémarrage.

Comment se détacher d'un conteneur sans l'arrêter ?

Ctrl+P Ctrl+Q



C'est une séquence de touches magique. Contrairement à `exit`, cette combinaison nous détache du conteneur mais laisse son processus principal (le shell) tourner en arrière-plan. C'est extrêmement utile pour les serveurs ou les tâches de fond.

Notre conteneur tourne-t-il toujours après le détachement ?

```
docker ps
```



```
$ docker ps
```

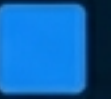
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a1b2c3d4e5f6	ubuntu	'bash'	5 minutes ago	Up 2 minutes		conteneur

Comme après `docker start`, la commande `docker ps`, nous montre que `conteneur` est toujours actif ('Up'). Nous avons réussi à le laisser tourner en tâche de fond tout en reprenant le contrôle de notre terminal hôte.

Phase 4 : Le Grand Nettoyage

Comment arrêter proprement un conteneur actif ?

```
docker stop conteneur
```



```
$ docker stop conteneur  
conteneur
```

La commande `docker stop` envoie un signal d'arrêt (SIGTERM) au processus principal du conteneur, lui permettant de se terminer proprement. Après quelques secondes, si le processus ne s'est pas arrêté, Docker envoie un signal SIGKILL pour le forcer.

Phase 4 : Le Grand Nettoyage

Comment supprimer un conteneur arrêté ?

```
docker rm conteneur
```



```
$ docker rm conteneur  
conteneur
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Une fois un conteneur arrêté, `docker rm` le supprime définitivement. Cette action est irréversible et efface le système de fichiers du conteneur ainsi que toutes les modifications que nous y avons apportées. On ne peut supprimer qu'un conteneur préalablement arrêté.

Comment supprimer l'image Docker qui ne nous sert plus ?



``rm i`` signifie 'remove image'. Cette commande supprime l'image ``ubuntu`` de notre machine. On ne peut supprimer une image que si aucun conteneur (même arrêté) ne l'utilise. C'est pourquoi nous devons faire ``docker rm`` d'abord.

Le Cycle est Complet

Notre environnement est revenu à son état initial.

Conteneurs :

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Images :

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

En relançant nos commandes d'état des lieux, nous pouvons constater que le conteneur et l'image ont tous deux disparu. Nous avons parcouru le cycle de vie complet : création, modification, exécution, arrêt et nettoyage. Votre système est aussi propre qu'au début.